Part A: Making the Fronthaul Lighter and the DU Faster

Structure-aware IQ compression, RMT-guided warm-start complex inversion, and a fused C/AVX-512 RZF precoder kernel for O-RAN 7-2x

Mayank Yadav Harshvardhan Choudhary

B.Tech. Project — Final Evaluation

November 2026

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. RMT-WS: Complex Matrix Inversion
4. Kernel Programming
5. Fronthaul Impact and Summary

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. RMT-WS: Complex Matrix Inversion
4. Kernel Programming
5. Fronthaul Impact and Summary

Why Part A? Two bottlenecks of a 5G/6G O-RAN cell

1. The fronthaul is huge

- Raw 64-antenna cell: ≈ 15.4 Gbit/s
- DU-side precoding, 100 MHz, 64 streams: **118 Gbit/s**
- O-RAN BFP only shortens numbers: $\approx 4\times$

$\Rightarrow$ compression that exploits channel structure

2. The DU has 0.5 ms per slot

- RZF per PRB: $G = HH^H + \alpha I$, $W = H^H G^{-1}$
- 273 PRBs $\times$ complex $K \times K$ inverse every slot
- numpy + LAPACK: **3.2 ms** for $K = 16$ ($6\times$ over budget)

$\Rightarrow$ fast, accurate complex inversion

Our answer: compress the data with its own structure, and invert the complex channel matrices with an RMT-guided method running on hand-optimised kernels.

Part A attacks both bottlenecks: fewer bits on the link, and a precoder that fits in the slot.

Our Part A contributions at a glance

CSEE & RAS-BFP

13–16$\times$ compression vs $\approx 4\times$ for O-RAN BFP; 3–6$\times$ faster than SVD

ACAFS

Rank-adaptive functional split: **55–75 %** bandwidth cut at high SNR

RMT-WS inversion

Complex Hermitian Grams: **6 GEMMs** vs > 10 for Neumann-type methods

Warm-start tracking

2.7$\times$ fewer GEMMs, **30**$\times$ lower loss at pedestrian speed; never diverges at 120 km/h

Fused C/AVX-512 kernel

Whole RZF precoder **10–15**$\times$ faster than numpy+LAPACK; fits the 0.5 ms slot

RU-side precoding

$-72\,\%$ fronthaul at 99.7 % of exact RZF sum-rate

From compression theory to a slot-feasible precoder: a complete DU-side toolchain.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 RMT-WS: Complex Matrix Inversion

4 Kernel Programming

5 Fronthaul Impact and Summary

Structure-aware IQ compression

$$Y = H \odot X + W \in \mathbb{C}^{M \times N}, \quad M = 64 \text{ antennas}, N = 1200 \text{ subcarriers, 3GPP TDL-A}$$

CSEE — delay-domain sparse encoder

1. IFFT along subcarriers $\rightarrow$ delay domain
2. Keep top-K taps (channel is sparse in delay)
3. Per-antenna BFP; FFT back at the receiver

Theorem 1: $\mathrm{NMSE} \leq \dfrac{\sigma_{\mathrm{tail}}^2}{\sigma_Y^2} + \dfrac{\Delta^2}{12}\dfrac{K}{MN\sigma_Y^2}$

Complexity $O(MN \log N)$

RAS-BFP — randomised sketch + QR

1. SRHT sketch $Z = \Omega Y \in \mathbb{C}^{r \times N}$
2. Thin QR $Z^H = QR$; project $L = YQ$
3. Send BFP(L), BFP(Q); $\quad Y \approx LQ^H$

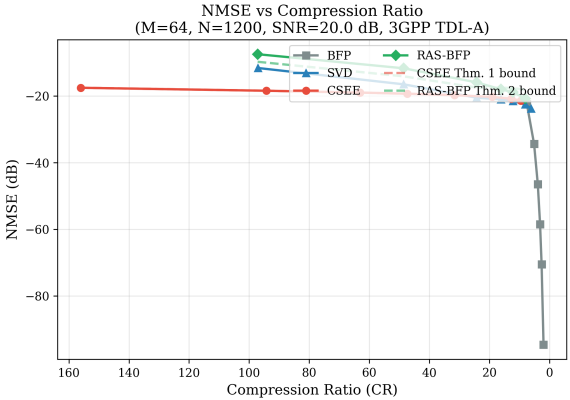
Theorem 2:
$\|Y - LQ^H\|_F^2 \leq (1 + \varepsilon_r)\|Y - Y_r\|_F^2$ w.h.p.
Theorem 4: $r^\star = \lfloor \sqrt{B_{\mathrm{total}}/(b(M+N))} \rceil$, $O(rMN)$

Both encoders come with provable error bounds and lower complexity than SVD.

Compression: 4× more than the O-RAN standard

$M = 64$, $N = 1200$, SNR 20 dB, TDL-A

Method	Compression	Encode
O-RAN BFP ($b = 8$)	3.9×	3.1 ms
Truncated SVD ($r = 12$)	16.2×	18–30 ms
CSEE (ours)	**12.7×**	**5.2 ms**
RAS-BFP (ours)	**16.2×**	**7.2 ms**



- 13–16× compression at ≈ -20 dB NMSE
- RAS-BFP: SVD-level compression, 3–4× faster
- CSEE: 4–6× faster than SVD; $\approx 63\times$ at the $K = 60$ knee
- Stable NMSE across SNR -5 to 30 dB

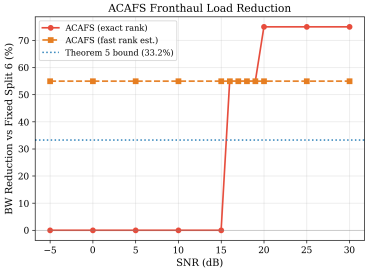
RAS-BFP matches SVD compression at a fraction of its encode time.

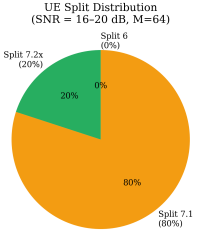
ACAFS: rank-adaptive functional split

Per-UE rank fraction r/M picks the O-RAN split:

Rank fraction r/M	Split
< 0.15	7.2x
$0.15 - 0.55$	7.1
≥ 0.55	6

- **Theorem 5:** closed-form bandwidth saving vs a fixed Split 6
- $\approx 33\,\%$ saving for uniformly spread ranks
- **55–75 %** saving at high SNR (rank collapse)
- Decision per UE, from the rank the encoder already computes





The split adapts to the channel instead of being fixed at deployment.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 RMT-WS: Complex Matrix Inversion

4 Kernel Programming

5 Fronthaul Impact and Summary

The real DU workload: complex RZF precoding

Realistic complex channels (`cmatinv/`)

- 3GPP TR 38.901 UMa NLOS path loss + shadowing (near-far)
- TDL-A 23-tap profile over 100 MHz
- Per-user angular-spread spatial correlation
- Rician line-of-sight option
- Clarke/Jakes aging $\rho = J_0(2\pi f_D T)$: 3–120 km/h
- $M = 64/128$, $K = 8/16/32$, 273 PRBs, 30 kHz, 3.5 GHz

Per PRB, every 0.5 ms slot

$$G = HH^H + \alpha I \in \mathbb{C}^{K \times K}$$

(complex Hermitian)

$$W = H^H G^{-1} \in \mathbb{C}^{M \times K}$$

$\times$ 273 PRBs inside one slot

Our question: can the whole complex precoder run inside the slot on a CPU DU, and can inversion exploit the channel's statistics?

RMT-WS: random-matrix-guided warm-start inversion

1. **Equilibrate** $\hat{G} = D^{-1/2} G D^{-1/2}$ — removes the near-far spread
2. **Eigensolve-free spectral interval** (Marchenko–Pastur + moment match):

$$v = \|\hat{G}\|_F^2/K - 1, \qquad [a, b] = 1 + v/\bar{s} \mp 2\sqrt{v} \quad (K^2 \text{ work})$$

3. **Cold start:** Chebyshev polynomial on $[a, b]$, then Newton–Schulz $X \leftarrow X(2I - \hat{G}X)$ (quadratic, pure GEMMs)
4. **Tracking:** next slot starts from the previous inverse
5. **Residual certificate** $\|(I - \hat{G}X)z\|/\|z\|$ picks the Newton steps per PRB, or falls back to a cold start — always safe

Why it fits the DU

- ✓ Adapts to correlated and near-far channels automatically
- ✓ No eigen-decomposition
- ✓ Only matrix multiplies: GPU / NPU / tensor-core friendly
- ✓ Cost adapts to user speed
- ✓ Built-in accuracy certificate

Random-matrix theory gives the inverse a head start; the certificate keeps it safe.

Cold-start inversion: fewer GEMMs

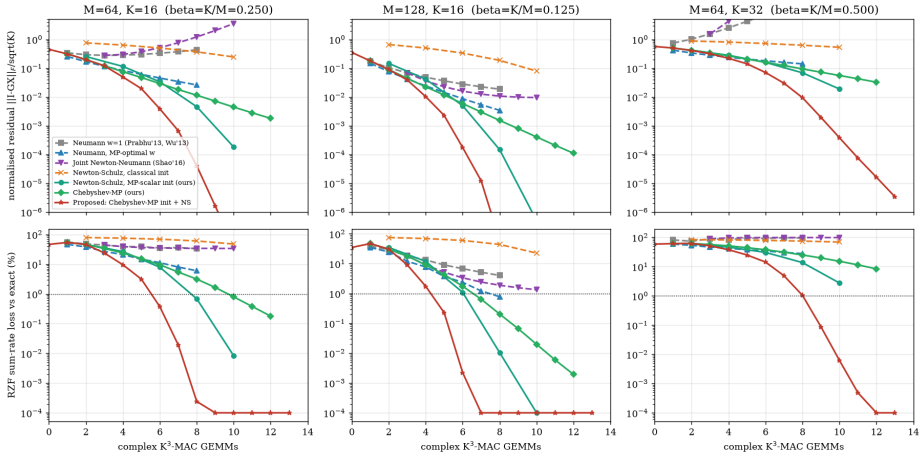
Complex GEMMs (K^3 MACs) needed for $\leq 1\,\%$ RZF sum-rate loss

Method	64/16	128/16	64/32
Neumann (Wu / Prabhu)	> 8	> 8	> 8
Joint Newton–Neumann (Shao)	> 10	> 10	> 10
Newton–Schulz, classical init	> 10	> 10	> 10
Block conjugate gradient	7	5	12
Gauss–Seidel	6	4	> 12
Cheb-MP + Newton–Schulz (ours)	**6**	**5**	**9**

- Beats every Neumann / Newton-type scheme in every configuration
- Best of all methods at heavy load ($K/M = 0.5$): **9 vs 12** GEMMs

Our initialisation converges in 6 GEMMs where classical series need more than 10.

Cold start: residual and sum-rate loss vs GEMM budget



Red curve (ours) reaches 1 % loss first in every antenna/user configuration.

Warm-start tracking across user speeds

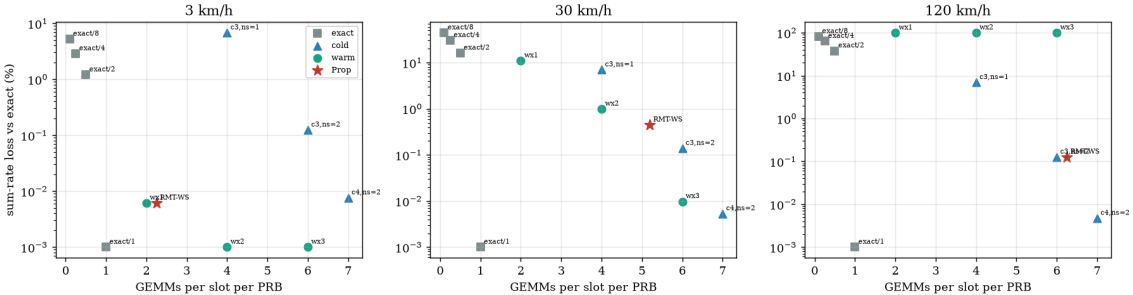
$K = 16$, $M = 64$; sum-rate loss vs exact RZF / GEMMs per slot per PRB

Speed	Warm Gauss–Seidel $\times 4$	Warm CG $\times 4$	**RMT-WS (ours)**
3 km/h	0.009 % / 4	0.013 % / 5	**0.006 % / 2.25**
30 km/h	0.63 % / 4	0.96 % / 5	**0.43 % / 5.2**
120 km/h	2.9 % / 4	5.5 % / 5	**0.12 % / 6.25**

- 3 km/h: **2.7× fewer GEMMs** and **30× lower loss** than a cold start
- 30 km/h: lowest loss of all warm-started methods
- 120 km/h: **24× lower loss** than warm Gauss–Seidel
- Fixed 1- or 2-step warm starts diverge at 120 km/h — RMT-WS never does

One method, every speed: cheap when users are slow, safe when they are fast.

Tracking: accuracy vs cost at 3, 30 and 120 km/h



RMT-WS (red star) sits at the low-cost, low-loss corner at every speed.

A closed-form speed law and a safety certificate

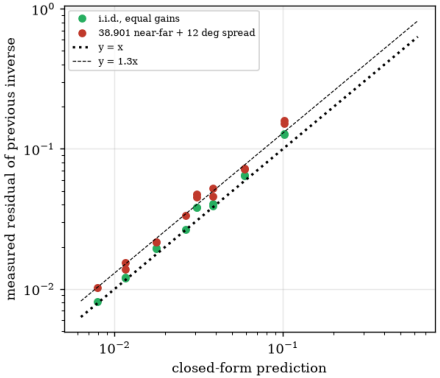
Doppler-to-warm-start law

$$r_0 \approx c \cdot 2\sqrt{\tfrac{K}{M-K}}\,\sqrt{1-\rho}, \qquad 1-\rho \approx (\pi f_D T)^2$$

- r_0 grows linearly with speed (measured slope 1.00)
- Predicts the Newton steps: $\lceil \log_2(\ln \varepsilon / \ln r_0) \rceil$
- Warm start valid up to $\approx$ **39 km/h** (64/16), 60 km/h ($K = 8$), 23 km/h ($K = 32$)

Certified safety

Tail bound $\Pr(\hat{r}^2 \le (1-d)r^2) \le e^{-pd^2 s/2}$
Audit of 480 PRB cases, 3–120 km/h: **0**
accepted-but-non-contractive at $\hat{r} \le 0.3$



A design rule: the DU knows in advance up to which speed the warm start pays off.

Robust at low precision

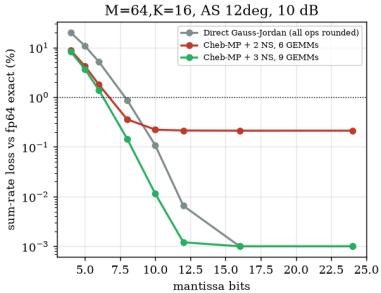
GEMM operands rounded to b mantissa bits, wide accumulate (tensor-core / NPU style)

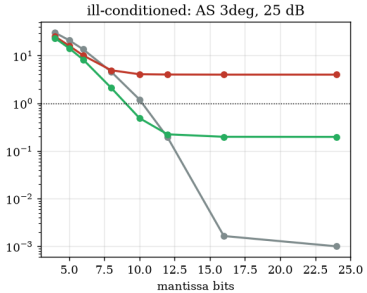
Setting	Direct G–J	**Ours (Cheb + 3 NS)**
8 mantissa bits	0.88 %	**0.15 %**
6 mantissa bits	5.1 %	**1.4 %**
Ill-cond., 10 bits	1.2 %	**0.49 %**

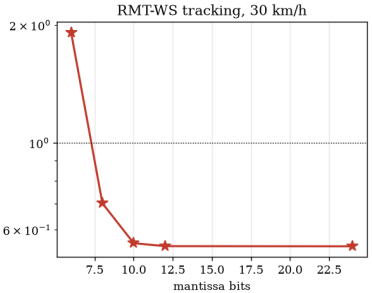
- Sum-rate loss vs float64 exact
- **6**$\times$ lower loss at 8 bits, **3.6**$\times$ at 6 bits
- Newton iteration self-corrects rounding errors
- Tracker at 8 bits: 0.70 % vs 0.54 % at full precision (30 km/h)

Our method is ready for low-precision accelerators where direct inversion degrades.

Low precision: loss vs mantissa bits







Newton refinement keeps the loss low where direct inversion degrades.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 RMT-WS: Complex Matrix Inversion

4 Kernel Programming

5 Fronthaul Impact and Summary

Kernel programming: a fused complex RZF precoder in C

`kernels/cmat_kernels.c` — batched complex64, AVX-512, OpenMP; binding `cmatinv/fastkernels.py`

- One pass per PRB: load H once $\rightarrow$ Gram $\rightarrow$ inverse $\rightarrow W^T$ + power scaling; stack / L1-resident, no DRAM intermediates
- Hermitian symmetry: Gram on the lower triangle only
- Split re/im planes so complex arithmetic vectorises
- In-place Gauss–Jordan sweep: K rank-1 updates, no sqrt/divide chains (inverse 6.4k $\rightarrow$ **2.9k** cycles/PRB, $K = 16$)
- Register-blocked $W = H^H X$: 0.81 $\rightarrow$ **0.42** ms/slot
- AVX-512 codegen: 1.3–1.5$\times$ single thread, 2$\times$ on 4 threads
- Fused Newton step returns the exact residual for free

```
// inner loop of the sweep inverse
for (int i = 0; i < K; i++) {
  if (i == k) continue;
  float fr = ckr[i]*piv, fi = cki[i]*piv;
  float *ar = xr + i*K, *ai = xi + i*K;
  #pragma omp simd
  for (int j = 0; j < K; j++) {
    ar[j] -= fr*rkr[j] - fi*rki[j];
    ai[j] -= fr*rki[j] + fi*rkr[j];
  }
  ar[k] = -fr; ai[k] = -fi;
}

gcc -O3 -march=native -fopenmp
    -mprefer-vector-width=512
```

Kernel results: the whole precoder now fits in the slot

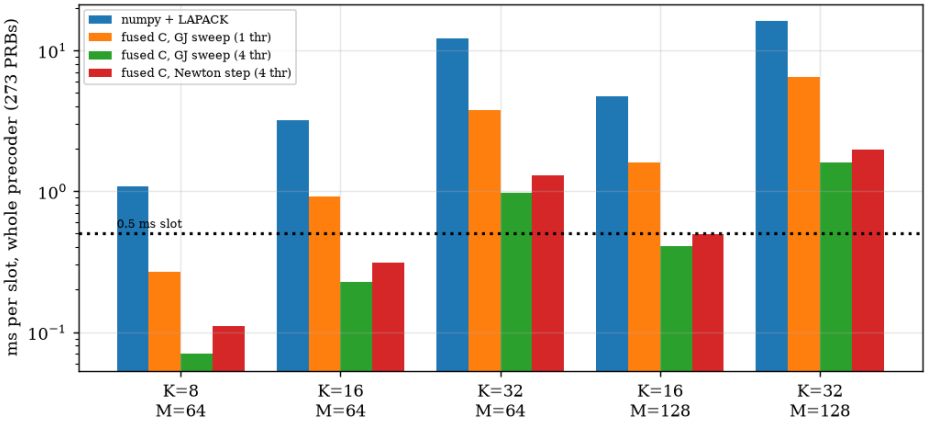
273 PRBs, complex64, 4-core AVX-512 Xeon, median of 40 runs

K, M	numpy+LAPACK	**Our kernel**	Speed-up
8, 64	1.08 ms	0.070 ms	15×
16, 64	3.21 ms	**0.226 ms**	**14×**
32, 64	12.2 ms	0.97 ms	12.5×
16, 128	4.71 ms	0.41 ms	11.6×
32, 128	16.3 ms	1.60 ms	10×

- $K = 8, 16$ ($M = 64$) and $K = 16$, $M = 128$ fit a 0.5 ms slot
- Near-linear thread scaling: 0.92 → 0.45 → 0.23 ms
- Accuracy: sum-rate loss $< 10^{-5}\,\%$ vs float64

10–15× faster: from 6× over the slot budget to under half of it.

Whole-precoder time per slot (273 PRBs)



Our fused kernels (green, red) run under the 0.5 ms line for $K \leq 16$; numpy+LAPACK never does.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 RMT-WS: Complex Matrix Inversion

4 Kernel Programming

5 Fronthaul Impact and Summary

Where should precoding run? A 72 % fronthaul saving

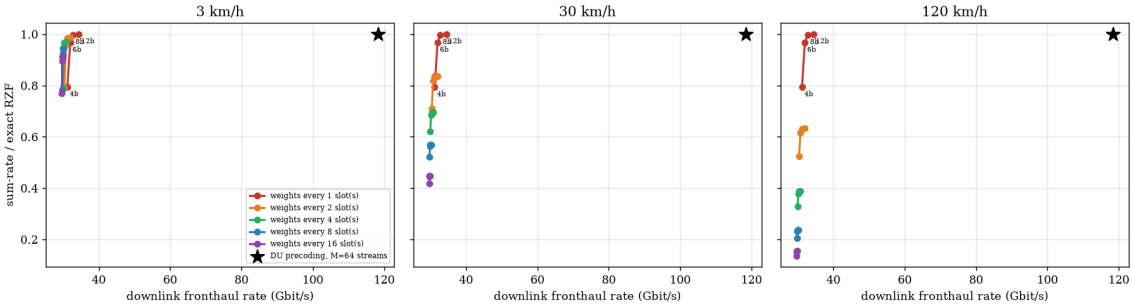
100 MHz, $M = 64$, $K = 16$, BFP-9 IQ

Architecture	Fronthaul	Sum-rate vs exact
DU precodes, sends 64 streams	118.4 Gbit/s	100 %
RU precodes, 16 layers + 8-bit weights	**32.8 Gbit/s ($-72\,\%$)**	**99.7 %**
RU precodes, 16 layers + 6-bit weights	32.0 Gbit/s ($-73\,\%$)	96.7 %

- Mobility requires refreshing the weights every slot (every 4 slots: only 70 % of the rate at 30 km/h)
- Per-slot weights are exactly what our fast kernel delivers
- RU-side precoding cuts the link load by **3.6×**

Fast per-slot precoding unlocks a 72 % lighter fronthaul at 99.7 % of the rate.

Fronthaul rate and sum-rate by architecture



RU-side precoding with per-slot weights: $-72\,\%$ fronthaul at 99.7 % rate.

Part A summary

Contribution	Headline result
CSEE / RAS-BFP compression	13–16× vs $\approx 4\times$ for O-RAN BFP; 3–6× faster than SVD; provable error bounds
ACAFS adaptive split	55–75 % bandwidth saving at high SNR
RMT-WS cold start	6 GEMMs vs > 10 for Neumann / Newton-type; best at $K/M = 0.5$
RMT-WS tracking	2.7× fewer GEMMs, 30× lower loss at 3 km/h; safe up to 120 km/h
Speed law + certificate	Closed-form warm-start envelope; 0 unsafe decisions in 480 cases
Low precision	6× lower loss than direct inversion at 8 bits
Fused C/AVX-512 precoder	10–15× faster than numpy+LAPACK; $K \leq 16$ fits the 0.5 ms slot
RU-side precoding	$-72\,\%$ fronthaul at 99.7 % of exact sum-rate

Part A delivers a lighter link and a slot-feasible precoder — the foundation for Part B.

Next: taking RMT-WS to GPU and tensor cores

- RMT-WS is pure matrix multiplication — the native operation of GPUs, NPUs and tensor cores
- $K = 16$ Gram matrices match a 16×16 tensor-core tile exactly
- Our low-precision results show the method keeps its accuracy at 8 bits
- Plan: CUDA port of the fused precoder; benchmark against cuBLAS / cuSOLVER batched
- Near-RT RIC deployment of ACAFS as an xApp
- Compression width as a per-cell knob for the Part B allocation rule

Thank you — questions?